

ANTAY FÁBRICA DE SOFTWARE

Reporte Cobranzas

Functional Requirements Document

Versión FRD v2.0

Versión app v1.7.3

Fecha 2026-03-15

Product Owner Camilo Ortega F.R.

Estado CRM TIER 1 completado — 141/141
WA tests ✓

Supabase Migración completa (MIG-000 → MIG-009)

FRD consolidado — Antay® 2026

FRD v2.0 – Reporte Cobranzas (Antay)

Proyecto: ReporteCobranzas – Cobranzas Antay

Product Owner: Camilo Ortega F.R.

Versión FRD: 2.0

Fecha: 2026-03-15

Versión app: v1.7.3

Estado Supabase: Migración completa (MIG-000 a MIG-009)

Estado CRM WA: TIER 1 completado (v1.6.0 → v1.7.3, 141/141 tests)

0. Glosario mínimo

Término	Definición
SSOT	Single Source of Truth — dataset maestro en memoria (<code>df_final</code>)
Vista filtrada	Dataset para pantalla según filtros activos (<code>df_filtered</code>)
Fresh Load / Ciclo nuevo	Carga nueva de los 2 Excel que reinicia el proceso
Ciclo	Unidad de procesamiento identificada con <code>cycle_id</code> (CIC-YYYYMMDD-HHMM)
Gate	Quality Gate — validación obligatoria antes de declarar un cambio "done"
E2E	End-to-End — prueba completa del flujo real del usuario
Rollback	Reversión al tag estable anterior
Cloud-only	Sin fallback local; si Supabase no responde → bloqueo controlado

1. Problema a resolver

Notificar a clientes por Email y/o WhatsApp usando un Reporte General construido desde 2 Excel externos:

1. Cuentas por Cobrar
2. Cobranzas

La cartera de clientes se mantiene en Supabase (no se recarga por Excel en cada ciclo).

La app debe:

- Generar el Reporte General (cliente + documentos por pagar)
 - Permitir enviar notificaciones por Email y WhatsApp
 - Mantener trazabilidad de envíos (quién, cuándo, con qué resultado)
 - Gestionar acuerdos de pago y seguimiento CRM
 - Permitir reiniciar el ciclo o reiniciar solo el tracking
-

2. Alcance

2.1 Incluye (IN)

- Carga de 2 Excel y generación de Reporte General
- Cartera maestra mantenida en Supabase (`clientes`)
- Tab Notificaciones Email: selección, preview HTML, envío, reporte post-envío
- Tab WhatsApp: envío masivo, 7 plantillas variables, seguimiento post-lote
- CRM Centro de Gestiones: registro de gestiones, acuerdos de pago, bandeja pendientes
- Trazabilidad completa (ciclos, reconciliación, resumen tablas)
- Persistencia de sesión con auto-restore del último ciclo
- Selector de ciclos históricos
- Módulo Clientes Premium (CRUD completo desde app)
- Ambientes PROD/STAGING con detección automática

2.2 No incluye (OUT)

- Rediseñar lógica financiera de deuda/detracción
- Cambiar nombres de columnas clave (prohibido – ver sección 3)
- Cambiar motor de PDF/exportación

- Fallback local (SQLite/session_state) – cloud-only desde v1.6

3. Reglas NO negociables (Anti-regresión)

1. **NO romper el flujo funcional existente.**
2. **NO inventar procesos sin actualizar este FRD.**
3. **NO renombrar columnas clave:**
 - `CodCliente` , `Empresa` , `SaldoReal` , `Correo` , `MATCH_KEY`
 - `ESTADO_EMAIL` , `FECHA_ULTIMO_ENVIO` , `ESTADO_WHATSAPP`
4. Mejoras UI/UX solo si el FRD no pide lógica nueva.
5. Todo cambio debe pasar Gates antes de declararse "done".
6. El agente ejecuta autónomamente análisis, implementación, pruebas y documentación. Solo pide confirmación ante ambigüedad real de regla de negocio no definida en el FRD.

4. Modelo de datos

4.1 Entradas (Excel – 2 archivos)

- **Archivo A:** Cuentas por Cobrar
- **Archivo B:** Cobranzas (Detalle)

La Cartera de clientes NO se carga por Excel en cada ciclo. Se mantiene en Supabase.

4.2 SSOT en memoria

- `df_final` – dataset maestro (SSOT)
- `df_filtered` – vista derivada según filtros activos del Reporte General

4.3 Columnas de envío

- `Correo` – valor crudo del Excel (o de Supabase `clientes.email`)
- `EMAIL_FINAL` – email destino normalizado (minúsculas + trim)
- Regla: la app trabaja por `CodCliente` pero envía a `EMAIL_FINAL`
- Un `EMAIL_FINAL` puede corresponder a múltiples clientes (ej: contabilidad compartida)
- **EMAIL_FINAL no se elimina ni cambia de rol**

4.4 Columnas de tracking (obligatorias)

Columna	Valores	Notas
ESTADO_EMAIL	PENDIENTE / ENVIADO / ERROR / SIN_CORREO	Se inicializa en Fresh Load
FECHA_ULTIMO_ENVIO	timestamp	Vacío hasta confirmación de envío
ESTADO_WHATSAPP	PENDIENTE / ENVIADO / ERROR / SIN_TELEFONO	Ídem

ESTADO_ENVIO_TEXTO es campo legacy opcional, no se inicializa en Fresh Load.

5. Flujo del usuario

5.1 Flujo principal (ciclo diario)

1. Cargar 2 Excel → se genera Reporte General → se crea `cycle_id`
2. Tracking inicial: `ESTADO_EMAIL = PENDIENTE` , `FECHA_ULTIMO_ENVIO = vacío`
3. Filtrar Reporte General → vista `df_filtered` se sincroniza
4. Ir a Tab Email → seleccionar cliente(s) → preview HTML → enviar
5. Ir a Tab WhatsApp → seleccionar plantilla → envío masivo → registrar resultado post-lote
6. Ir a CRM → registrar gestiones, acuerdos, bandeja pendientes
7. La sesión persiste (al día siguiente se ve igual)

5.2 Nuevo ciclo (reemplaza todo)

- Botón "Cargar nuevos archivos" → confirmación 2 pasos
- Al confirmar: se elimina Reporte anterior, tracking vuelve a estado inicial
- Ciclo anterior queda en Supabase, recuperable por selector

5.3 Reiniciar solo tracking (Email)

- En Configuración → "Reiniciar Registro de Envíos"
- Acción: `ESTADO_EMAIL = PENDIENTE` , `FECHA_ULTIMO_ENVIO = vacío`

5.4 Auto-restore

- Al abrir la app: `attempt_auto_restore()` carga automáticamente el último ciclo
 - Flag `skip_auto_restore` evita que el auto-restore sobrescriba una elección manual
-

6. Reglas de conteo (críticas)

- "Enviados Hoy" y "Pendientes" se miden por `CodCliente` único, NO por EMAIL_FINAL
 - Si un cliente tiene fila Envío WA + fila Gestión → solo se cuenta una vez (deduplicar)
 - Monto multimoneda: guardar `DeudaS` / `DeudaD` explícitos en `wa_details` al enviar
 - Fallback: recalcular desde `df_filtered` agrupado por moneda + `CodCliente`
-

7. UI/UX

7.1 Reporte General

- Vista Ejecutiva / Vista Completa
- Pantalla completa real (preserva sesión)
- KPIs en parte superior: Total Saldo (S/ + \$), Total Detracción (solo S/), Total Clientes, Pendientes de Notificar

7.2 Tab Notificaciones Email

- KPIs: "Pendientes de Envío" y "Enviados Hoy" (por `CodCliente`)
- Reporte Post-Envío: obligatorio, persiste entre tabs, tabla con cliente/email/docs/resultado + resumen métricas

7.3 Tab WhatsApp

- Selector de plantilla antes del envío
- Sub-tabs de seguimiento por índice entero (`wa_subtab_idx`) – inmune a cambio de emoji en label
- Panel post-envío: monto `S/ X + $ Y` (no el doble)

7.4 Sidebar

- Banner STAGING visible cuando `SUPABASE_URL` contiene URL de staging
- Cabecera con pill "Enterprise" + versión actual
- Ciclo activo: ID legible + timestamp + filas
- Botón "Cambiar ciclo" para volver al selector sin recargar archivos

8. Persistencia con Supabase (cloud-only desde v1.6)

8.1 Tablas principales

Tabla	Tipo	Descripción
<code>clientes</code>	MAESTRA	Lista oficial. Se carga 1 vez desde Excel, luego se edita desde app
<code>documentos</code>	TRANSACCIONAL	Facturas/boletas. Se recarga en cada ciclo de Excel
<code>cobranzas</code>	TRANSACCIONAL	Aplicaciones (detracciones, amortizaciones, pagos)
<code>notificaciones</code>	TRANSACCIONAL CRÍTICA	Registro permanente de cada envío. NUNCA se borra
<code>gestiones</code>	TRANSACCIONAL	Gestiones CRM (registros de contacto, resultado)
<code>acuerdos_pago</code>	TRANSACCIONAL	Acuerdos de pago registrados
<code>cuotas_acuerdo</code>	TRANSACCIONAL	Cuotas por acuerdo
<code>ciclos_procesamiento</code>	TRANSACCIONAL	Metadata de cada ciclo cargado
<code>resumen_cliente_ciclo</code>	ANALÍTICA	1 fila por cliente por ciclo
<code>resumen_ciclo</code>	ANALÍTICA	1 fila por ciclo con totales
<code>app_config</code>	CONFIGURACIÓN	Plantillas WA, config SMTP, parámetros

8.2 Reglas por tabla

- `clientes` : NO se recarga en cada ciclo

- `notificaciones` : NUNCA se borra, se acumula historial
- `ciclos_procesamiento` : se acumula, cada ciclo es identificable y recuperable
- `cycle_id` formato: `CIC-YYYYMMDD-HHMM`

8.3 Reconciliación de tracking

Al restaurar ciclo X: cruzar `df_final` con `notificaciones WHERE cycle_id = X` para reconstruir `ESTADO_EMAIL` y `FECHA_ULTIMO_ENVIO` sin depender de memoria.

8.4 Criterios de aceptación Supabase

- CA-SUP-1: Cargar Excel → datos persisten en Supabase
- CA-SUP-2: Enviar email → registro aparece en `notificaciones`
- CA-SUP-3: Recargar Excel → notificaciones previas NO se pierden
- CA-SUP-4: Consultar "enviados ayer" → resultado correcto
- CA-SUP-5: Editar cliente desde app → cambio persiste
- CA-SUP-6: Selector de ciclos → carga ciclo correcto con tracking reconciliado

9. CRM WhatsApp – TIER 1 (v1.6.0, completado 2026-03-13)

RC-FEAT-019: Panel Resultado Post-Envío WA

- Panel de seguimiento post-lote en Tab WhatsApp
- Opciones: `EXITOSO` / `PROMETIO_PAGAR` / `SIN_RESPUESTA` / `ESCALAR`
- Llama a `insert_gestion()` con `tipo_gestion=WHATSAPP`
- Persiste en `gestiones.resultado` en Supabase

RC-FEAT-020: Biblioteca 7 Plantillas WhatsApp

- Selector visual antes del envío masivo
- 7 plantillas: primer aviso, recordatorio, aviso firme, acuerdo, pre-legal, felicitación, solicitud datos
- Variables: `{empresa}` , `{monto}` , `{fecha_venc}` , `{gestor}` , `{PROX_VENC}`
- Editables desde Tab Configuración, guardadas en `app_config`

RC-FEAT-021: Módulo Acuerdos de Pago con Cuotas

- Tablas: `acuerdos_pago` + `cuotas_acuerdo` en Supabase
- Formulario: cliente, monto total, nro cuotas, fecha inicio

- Cálculo automático de fechas de vencimiento
- Timeline visual: `PENDIENTE` / `PAGADA` / `VENCIDA`
- WA automático de confirmación al crear acuerdo

RC-FEAT-022: Bandeja de Pendientes del Día

- Lista priorizada: `URGENTE` / `ALTO` / `MEDIO`
- Detecta: WA sin respuesta +48h, cuotas venciendo ≤ 3 días, clientes +30 días mora sin gestión
- Botón de acción directa por ítem

RC-FEAT-023: Trazabilidad Completa

- Tablas: `resumen_cliente_ciclo` + `resumen_ciclo`
- `reconcile_ciclo_recovery()` en `db_manager.py`
- Documentos desaparecidos → estado `RECUPERADO` + fecha + forma_pago + banco

RC-FEAT-024: Tabs CRM Persistentes al Preparar Nuevo Ciclo

- Sidebar no limpia `df_final` al confirmar reemplazar archivos
- CRM sigue visible durante la transición

RC-FEAT-025: Auto-restore Último Ciclo

- `attempt_auto_restore()` en `app.py` al abrir la app
 - Flag `skip_auto_restore` evita sobreescritura por elección manual del gestor
-

10. Hotfixes post-TIER 1 (v1.7.0 → v1.7.3)

ID	Descripción	Fix
RC- BUG- 024	<code>CREATE POLICY IF NOT EXISTS</code> incompatible con PostgreSQL	Bloque <code>DO \$\$</code> en sql/11, sql/12
RC- BUG- 025	<code>insert_acuerdo_pago</code> error <code>.select()</code> encadenado	supabase-py sync no lo soporta — eliminado
RC- BUG- 026	<code>ESTADO_EMAIL</code> / <code>ESTADO_WHATSAPP</code> en blanco al restaurar	<code>fillna('PENDIENTE')</code> en <code>_docs_to_df()</code>
RC- BUG- 027	Selectbox plantilla WA se resetea en cada rerun	<code>key="wa_plantilla_seleccionada"</code> fijo
RC- BUG- 028	Variable <code>{PROX_VENC}</code> no disponible en plantillas	Agregada a <code>contact_data</code> y preview
RC- BUG- 029	Botón "Cambiar ciclo" sobrescrito por auto-restore	Flag <code>skip_auto_restore</code> en session_state
RC- BUG- 030	Sub-tab Seguimiento reseteaba al tab 1 en cada rerun	Persistir por índice entero <code>wa_subtab_idx</code>
RC- BUG- 031	Monto WA mostraba solo S/ y doble conteo	Guardar <code>DeudaS</code> / <code>DeudaD</code> explícitos; deduplicar <code>CodCliente</code>
RC- UX- 013	Banner STAGING/PROD en sidebar	Detección automática via <code>SUPABASE_URL</code>
RC- OPS- 006	Ambiente staging configurado	Supabase staging + <code>.env.staging</code>

11. Módulo Clientes Premium (v1.7.1)

- Tab `Clientes Premium` : única superficie de mantenimiento de cartera maestra
- CRUD completo: agregar, editar, desactivar clientes desde la app
- Migración desde Excel con reporte de errores
- Home operativo estricto: solo acepta `CtasxCobrar` + `Cobranza` (sin uploader de Cartera)
- Si no hay cartera maestra → ciclo bloqueado con instrucción clara hacia Clientes Premium

12. Quality Gates (obligatorio antes de "done")

Gate	Descripción
Gate 0	Compilación: <code>py_compile</code> pasa sin errores
Gate 1	Tests automatizados: <code>pytest</code> para funciones críticas
Gate 2	Preflight: config/QA mode correcto, no enviar a reales en QA
Gate 3	Smoke manual con evidencia: screenshots/video de CA-1..CA-5 en staging
Gate 4	Documentación: actualizar FRD + changelog + notas de release

13. Criterios de aceptación globales

ID	Descripción
CA-1	Fresh Load → KPIs Email: Enviados=0, Pendientes>0. Tracking inicial PENDIENTE
CA-2	Filtrar Reporte → Tab Email refleja mismo subconjunto. Preview HTML = docs filtrados
CA-3	Cientes deuda 0 no aparecen para email (salvo detracción pendiente)
CA-4	Emails compartidos: no bloquea selección, KPIs por CodCliente
CA-5	Post-envío: tracking actualizado, KPIs actualizados, Reporte Post-Envío persiste

14. Control de cambios – Changelog

Versión	Fecha	Cambio
v0.1	2025-12-22	Versión inicial — Email + tracking básico
v0.2	2026-01-15	Estabilización ciclo Email + tracking + UX mínima
v0.3	2026-02-15	Integración Supabase cloud-only, ciclos persistentes
v1.6.0	2026-03-13	TIER 1 CRM WhatsApp completo (141/141 tests)
v1.7.1	2026-03-13	Módulo Clientes Premium + Home 2 archivos + hotfixes SQL
v1.7.2	2026-03-14	Mejoras CRM flow + auto-restore + banner STAGING
v1.7.3	2026-03-15	Fix sub-tab seguimiento WA (RC-BUG-030/031)
v2.0 FRD	2026-03-15	Consolidación FRD completo en repo local

15. TIER 2 – Próximo sprint (pendiente)

15.1 Operativo inmediato (v1.8.x)

Ticket	Descripción	Prioridad
—	Smoke test TIER 2 en staging (localhost:8502): panel post-envío WA, acuerdos, bandeja pendientes, banner	P0
—	Merge <code>dev → main</code> cuando staging esté verde	P0
RC-FEAT-026	Panel envío WA de prueba en Tab Configuración (smoke sin datos reales)	P1

RC-FEAT-026 – Panel WA de Prueba en Tab Configuración

- **Objetivo:** Permitir al gestor enviar un WA de prueba desde Tab Configuración sin necesitar datos reales cargados
- **UI:** Input teléfono (default `+51921566036`), textarea mensaje, botón "Enviar prueba"
- **Llama a:** `send_whatsapp_messages_direct()` con contacto ficticio
- **Archivo:** `utils/ui/tabs/config_tab.py` – dentro de sección WA, después del panel de sesión activa
- **CA:** Envío exitoso → toast verde; error → mensaje claro con causa

15.2 Features TIER 2 (v1.9.x)

Ticket	Descripción	Estimado	Prioridad
RC-FEAT-027	Selección automática de plantilla por Aging (días de mora)	~2h	P2
RC-FEAT-028	KPIs Expandidos de Efectividad de Cobranza	~2h	P2

RC-FEAT-027 – Selección Automática de Plantilla por Aging

- **Objetivo:** Al abrir Tab WhatsApp, asignar automáticamente la plantilla sugerida según días de mora del cliente
- **Regla de negocio:**

Rango	Segmento	Plantilla sugerida
0–14 días	Deuda reciente	Primer Aviso
15–30 días	Sin respuesta	Recordatorio
31–60 días	Mora significativa	Aviso Firme
60+ días	Mora crítica	Pre-Legal

- **El gestor siempre puede sobreescribir** antes de enviar (control humano sobre mensajes críticos)
- **Muestra visualmente** el segmento de mora de cada cliente en la tabla de selección
- **Archivo:** `utils/ui/tabs/whatsapp.py`
- **CA:** Segmento visible por cliente; plantilla pre-seleccionada pero editable; sin envío automático

RC-FEAT-028 – KPIs Expandidos de Efectividad de Cobranza

- **Objetivo:** Panel de métricas cruzadas en Tab WA y Centro de Gestiones
- **Métricas:**
 - Mensajes WA enviados hoy / esta semana
 - Con respuesta (EXITOSO + PROMETIO_PAGAR) vs Sin respuesta (SIN_RESPUESTA + ESCALAR)
 - Acuerdos de pago activos
 - Cuotas venciendo en ≤ 3 días
 - Monto total gestionado (S/ + \$) vs monto con acuerdo formal
- **Fuente de datos:** Tablas `gestiones`, `acuerdos_pago`, `cuotas_acuerdo`, `resumen_ciclo`
- **Archivos:** `utils/ui/tabs/whatsapp.py`, `utils/ui/tabs/crm_gestiones.py`, `utils/db_manager.py`
- **CA:** KPIs se calculan en tiempo real desde Supabase; sin afectar `df_final` ni `df_filtered`

16. TIER 3 – Features futuras (v2.x)

Estas features provienen de la Propuesta CRM WhatsApp v1.0 (documento de origen, ahora obsoleto). Se documentan aquí como roadmap aprobado para cuando TIER 2 esté completo.

Ticket	Descripción	Estimado	Valor
RC-FEAT-029	Registro de Pagos en Tiempo Real (sin esperar ERP)	~4h	MEDIO-ALTO
RC-FEAT-030	Dashboard de Efectividad de Cobranza (analytics)	~6h	ALTO

RC-FEAT-029 – Registro de Pagos en Tiempo Real

- **Problema que resuelve:** El gestor sabe que un cliente pagó pero el ERP no refleja el pago hasta el día siguiente (o días después). Hoy no puede registrar ese pago en la app.
- **Solución:** Formulario en CRM para que el gestor registre un pago recibido directamente en la app
- **Campos:** Cliente, monto, moneda, fecha, forma de pago, banco, referencia, nota
- **Efecto:**
- Actualiza `documentos.monto_pendiente` temporalmente (hasta que el ERP confirme)

- Marca cuota del acuerdo como `PAGADA` si corresponde
- Genera WA de agradecimiento automáticamente (plantilla "Felicitación")
- Marca el registro como "pendiente de confirmación ERP" para no confundir con datos reales
- **Riesgo:** Potencial desincronización con ERP — el registro es provisional hasta que el próximo Excel confirme
- **Archivos:** `utils/ui/tabs/crm_gestiones.py`, `utils/db_manager.py`

RC-FEAT-030 — Dashboard de Efectividad de Cobranza

- **Objetivo:** Reportes analíticos para el supervisor / dirección
- **Métricas objetivo:**
 - % de WA que resultan en pago (ventanas 7 / 15 / 30 días)
 - Ranking de clientes por dificultad de cobranza (gestiones sin resultado positivo)
 - Saldo total gestionado vs saldo recuperado (por ciclo y acumulado)
 - Evolución mensual de recuperación
 - Acuerdos cumplidos vs incumplidos
 - Tasa de respuesta por plantilla WA (qué plantilla convierte más)
- **Arquitectura:** Nuevo tab "Analytics" — consultas sobre `resumen_cliente_ciclo`, `resumen_ciclo`, `gestiones`, `acuerdos_pago`
- **Prerequisito:** RC-FEAT-023 (Trazabilidad) debe estar en producción ≥ 2 ciclos para tener datos suficientes
- **Archivo:** Nuevo `utils/ui/tabs/analytics.py`

17. Ambientes

Ambiente	Puerto	Supabase	Arranque
PROD	8501	proyecto PROD	<code>streamlit run app.py -server.port 8501</code>
STAGING	8502	<code>hrnqngndnohkkgztzgig.supabase.co</code>	ver <code>.env.staging</code>

Detección automática: Si `SUPABASE_URL` contiene la URL de staging → banner visible en sidebar.

FRD REPORTECOBRANZAS V2.0 – ANTAY

Apéndice UI/UX Diseños y Wireframes

Visualización de features pendientes – TIER 2 y TIER 3

TIER 2

2

Features

TIER 3

2

Features

PÁGINAS DE DISEÑO

4

Wireframes

TIER 2

RC-FEAT-027 – Selección Automática de Plantilla por Aging

Al abrir Tab WhatsApp, el sistema sugiere automáticamente la plantilla según días de mora de cada cliente. El gestor puede sobrecribir antes de enviar.

Regla de segmentación por días de mora



Wireframe – Tab WhatsApp con columna Segmento y plantilla pre-seleccionada

Tab: Marketing WhatsApp – Seleccion de Clientes

Buscar cliente...

Plantilla sugerida por Aging

Seleccionar todos

Check	Cliente	Telefono	Saldo Real	Dias Mora	Segmento	Plantilla Sugerida
[x]	COMERCIAL LIMA S.A.	+51 987- 654- 321	S/ 12,400	68 dias	Mora critica	Pre-Legal
[x]	INVERSIONES NORTE	+51 912- 345- 678	S/ 8,750	45 dias	Mora significativa	Aviso Firme
[x]	DISTRIBUIDORA SUR	+51 945- 678- 901	\$ 3,200	18 dias	Sin respuesta	Recordatorio
[]	SERVICIOS ANDINOS	+51 956- 789- 012	S/ 2,100	8 dias	Deuda reciente	Primer Aviso

3 clientes seleccionados · Monto total: S/ 21,150 + \$ 3,200

Enviar WA (3 clientes)



Control humano siempre presente: el sistema *sugiere* la plantilla pero el gestor puede cambiarla cliente por cliente antes de enviar. Nunca hay envio automatico.

utils/ui/tabs/whatsapp.py

~2 horas

Solo logica Python – Sin cambios BD

TIER 2

RC-FEAT-028 — KPIs Expandidos de Efectividad de Cobranza

Panel de metricas cruzadas visible en Tab WA y Centro de Gestiones. Calculado en tiempo real desde Supabase.



 **Sin impacto en flujo principal:** todos los KPIs se calculan con consultas directas a Supabase. No modifica `df_final` ni `df_filtered`.

utils/ui/tabs/whatsapp.py utils/ui/tabs/crm_gestiones.py utils/db_manager.py

gestiones · acuerdos_pago · cuotas_acuerdo · resumen_ciclo ~2 horas

TIER 3

RC-FEAT-029 – Registro de Pagos en Tiempo Real

El gestor registra un pago recibido directamente en la app sin esperar el ERP. El registro es provisional hasta que el siguiente ciclo Excel lo confirme.

Tab: Centro de Gestiones – Registrar Pago Recibido

Registrar pago provisional

Cliente *

COMERCIAL LIMA S.A.

Fecha de pago *

14/03/2026

Monto *

S/ 8,000.00

Forma de pago *

Transferencia bancaria

Banco

BCP

Referencia

TRF - 2026031412345

Nota del gestor

Cliente confirmo abono parcial por WhatsApp...

☒ Enviar WA de agradecimiento automáticamente (plantilla Felicitacion)

Cancelar

Registrar pago provisional

Estado actual del cliente

Saldo pendiente

S/ 12,400

Dias mora

68 dias

Registro provisional

Marcado como **pendiente de confirmacion ERP** hasta que el siguiente ciclo Excel lo confirme.

Acuerdo activo:

+

01/03

S/ 4,100

!

15/03

S/ 4,150

O

30/03

S/ 4,150

Riesgo documentado: si el siguiente ciclo Excel llega tarde o no llega, el registro provisional permanece. Se muestra indicador visual diferenciador en toda la UI.

utils/ui/tabs/crm_gestiones.py

utils/db_manager.py

documentos (monto_pendiente + flag provisional)

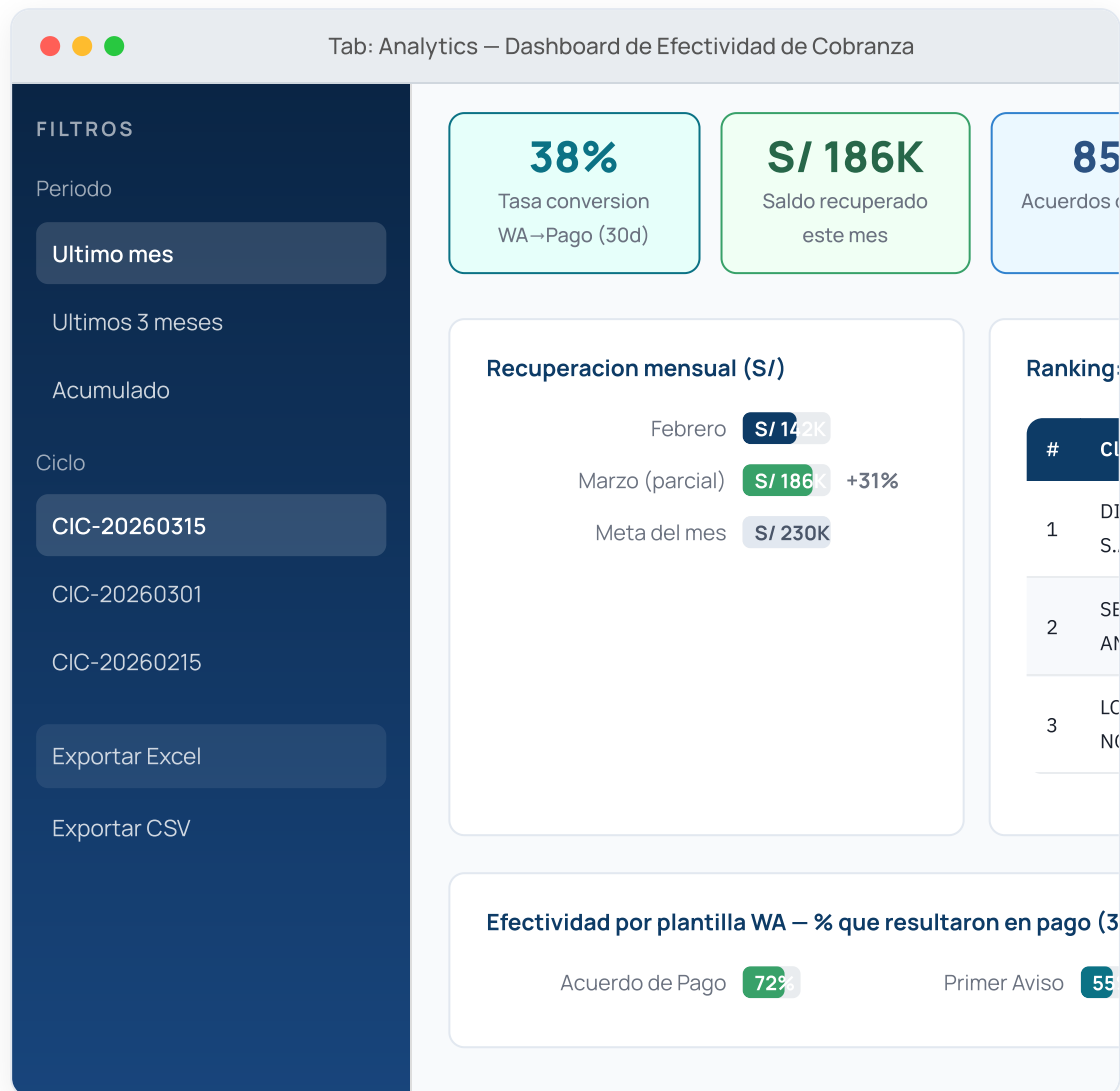
cuotas_acuerdo (estado PAGADA)

~4 horas

21

TIER 3 RC-FEAT-030 — Dashboard de Efectividad de Cobranza

Nuevo Tab Analytics con reportes ejecutivos para supervisor/direccion. Requiere 2+ ciclos en produccion para tener datos suficientes.



Prerequisito: RC-FEAT-023 (Trazabilidad) debe tener 2+ ciclos en produccion. Los datos vienen de `resumen_cliente_ciclo` y `resumen_ciclo`.

`utils/ui/tabs/analytics.py` (nuevo)

`utils/db_manager.py`

`resumen_cliente_ciclo` · `resumen_ciclo` · `gestiones` · `acuerdos_pago`

~6 horas